

## Project Specification: Member Management System

Technical breakdown of *docs/kiowa\_requirements.md*. Tasks are ordered by implementation dependency, not by requirement number — build top to bottom. A "Removed from scope" section at the bottom lists items from a previous draft that don't trace back to an actual client requirement.

Legend: [x] done · [ ] not started · [~] partially implemented (see note)

---

### Phase 1 — Contact Statuses

Requirement #4. Foundational: Communications' recipient grouping (Phase 3) depends on this.

- ☒ Replace `members.status` (`lib/schema.ts`) — currently `active/inactive` only — with three categories: **Waiting List**, **Non-Member**, **Member**.
- ☒ CSV export of the contact list (for mailing labels).

Board-side viewing/editing of the contact list already exists (`components/admin/MembersAdmin.tsx`, `app/api/admin/members`) and just needs the schema change above layered on top — no new UI shell required.

---

### Phase 2 — Stripe Integration

Requirement #5.d. Blocks Membership Form payment (Phase 3) and Member Portal payment (Phase 4). No Stripe integration exists anywhere in the codebase yet — build from scratch.

- ☒ Stripe Checkout for dues.
  - ☒ Support non-card payment methods (not just credit/debit).
  - ☒ Board-facing payment analytics: confirmation of amount/date per member.
  - [~] Surface whether a member provided a discount-card image, alongside their payment, so the board can cross-check reduced payments.
    - `documents.memberId` and a `discount_card` document category now exist, and the admin Payments view cross-checks them — but nothing sets `memberId` on a document yet, since self-service uploads don't exist until Phase 3.
- 

### Phase 3 — Membership Form

Requirement #5. Public renewal/waiting-list application form. `app/membership/page.tsx` is currently a static info page with PDF downloads only — no submission form exists yet.

- ☒ **Membership document** — display the Range Rules text/fields, with an acknowledgement checkbox and a digital signature method.
  - ☒ **Member info** — collect email, phone, home address.
    - `email/phone` already exist on the `members` table (`lib/schema.ts`); `address` column is missing.
    - Members are currently entered manually by an admin (`app/api/admin/members/route.ts`) — this form is the first self-service submission path.
-

- ☒ **Document uploads** — NRA membership proof, cleanup-day discount card, background-check cover page or CCL, plus a link to [criminalwatchdog.com](https://criminalwatchdog.com). Flagged for board review/approval, which updates the contact's status (Phase 1).
  - Reuse the existing R2 upload plumbing built for admin-managed documents ([app/api/documents](#), [components/admin/DocumentsAdmin.tsx](#)) rather than building a new uploader.
- ☒ **Payment** — Stripe checkout for dues (depends on Phase 2).

---

## Phase 4 — Communications

*Requirements #6 and #4.d.*

- ☒ Email, sent through Postmark ([lib/email.ts](#), [app/api/admin/email/route.ts](#)).
- ☒ Text messaging — SignalWire. Waiting on regulatory-compliance checks to pass.
- ☒ Automated renewal reminder texts, 45 and 15 days before a member's [renewal\\_date](#) ([app/api/cron/renewal-reminders/route.ts](#)), triggered daily by a companion Cloudflare Worker ([../kiowa-gun-cron](#)) since OpenNext's Worker has no native Cron Trigger support. All other messages stay manual (SHAFT/carrier compliance).
  - Route logic verified working end-to-end locally (correct 45/15-day threshold selection, SignalWire send, [renewal\\_45/15\\_reminder\\_sent\\_for](#) tracking, [smsCampaigns](#) audit log all confirmed 2026-08-08). Not live yet: [kiowa-gun-cron](#) has never been deployed ([wrangler secret list](#) returns "Worker not found"), and the [kiowa-gun](#) Worker's deployed secrets are missing [SIGNALWIRE\\_\\*/CRON\\_SECRET](#) (only [BREVO\\_API\\_KEY/RESEND\\_API\\_KEY](#) are set in prod). Needs [wrangler secret put](#) on [kiowa-gun](#) + [wrangler deploy](#) on [kiowa-gun-cron](#) before this actually fires in production.
- ☒ Embedded images in the composer — [RichTextEditor](#) uploads via [app/api/admin/email/images](#) for a live preview while editing; at send time [lib/emailAttachments.ts:buildEmailAttachments](#) rewrites those references into inline CID attachments (image bytes travel inside the email itself) instead of recipients' mail clients fetching a public URL.
- ☒ True file attachments (non-image, e.g. PDFs) — [EmailAdmin](#) has an "Attach a file" control ([app/api/admin/email/attachments](#)) for real downloadable Postmark attachments, separate from inline images; uploaded objects are deleted from R2 right after sending since they're one-shot.
- ☒ Button tool in the composer — [useLinkEditor.tsx:startButton](#) inserts an inline-styled CTA-style link, reusing the same in-place URL-editing popover as the existing Link tool.
- ☒ MMS picture attachments for texts — [SmsAdmin](#) has an "Attach a picture" control ([app/api/admin/sms/media](#), served at [app/api/sms-media/\[key\]](#)); [lib/sms.ts](#) passes it through to SignalWire as [MediaUrl](#). Plain links need no special handling in SMS (phones auto-linkify URLs); [sms\\_campaigns.mediaUrl](#) (migration [0019](#)) tracks it in send history.
- ☒ Attachment guardrails — combined inline-images + file-attachments budget capped at 6MB raw ([MAX\\_TOTAL\\_ATTACHMENT\\_BYTES](#), checked via R2 [HEAD](#) before downloading any bytes) to stay under Postmark's 10MB total-message cap; max 5 files per send; file-attachment [r2Key](#) refs and the SMS [mediaUrl](#) are validated server-side to stay inside our own upload namespaces (client-supplied refs/URLs are otherwise untrusted).

- ☒ Recipient grouping by status — `EmailAdmin` now has a status-group checklist (Waiting List / Non-Member / Member) that filters the recipient pool; `RecipientPicker` still handles search/select within the chosen groups.
  - ☐ Send analytics — opens, bounces, clicks, spam — broken out per recipient. Campaigns currently only log aggregate sent/failed counts (`emailCampaigns` table).
- 

## Phase 5 — Member Portal

*Requirement #4.c: members/visitors edit their own info via their own login. Reuses the Membership Form (Phase 3) for requirement #5.a ("used at any time to update personal info"). No member-facing auth exists yet — only admin/board login (`Lib/auth.ts`).*

- ☐ Login — members/applicants choose their own username and password.
  - ☐ Update password.
  - ☐ General info form — same form as Phase 3, reused behind login rather than rebuilt.
  - ☐ Update payment method for recurring yearly dues. **Depends on Phase 2.**
- 

## Phase 6 — Match Dates & Results

*Requirement #3. Independent of the other phases. A single-discipline schedule already exists (`app/matches/page.tsx`, `app/admin/(protected)/matches/`, `Lib/schema.ts:matches`) and needs discipline-grouping and per-match photo galleries layered on top.*

- ☒ **a.** Text displaying a match date is editable to serve as a Practiscore results link — `matches.resultsUrl`, editable via the admin Matches page; falls back to "(results not yet posted)" until filled in.
  - ☒ **b.** Board members can post/edit match info and the results link themselves, via the existing admin Matches CRUD (`app/api/admin/matches`, `components/admin/MatchesAdmin.tsx`).
  - ☒ **c.** Match Date pages need space for pictures the board can change — `matches.id`-scoped `match_photos` table (R2-backed, same upload pattern as calendar event images) replaces the 4 hardcoded gallery images; board uploads/deletes per match date via a "Pictures" panel in `components/admin/MatchesAdmin.tsx`, served publicly via `app/api/match-photos/[id]`.
  - ☒ Separate lists by Discipline — `matches.discipline` column (free text, defaults to "Defensive Pistol" so existing rows aren't orphaned) plus grouped rendering (one schedule table + results list per discipline, in first-appearance order) on both `app/matches/page.tsx` and `components/admin/MatchesAdmin.tsx`.
-